

Searching and sorting

Revise 360 lesson plan — 60 minutes

Unit	Beyond OCR J277: a Python pathway supporting 2.1, 2.2 and 2.3
Lesson	Searching and sorting (lesson 11 of the unit)
Lesson ID	pr-l11
Experience ID	pr-l11
Plan ID	PLAN-PR-L11
Content version	d7616973762e
Website	https://revise360.co.uk/experience.html?id=pr-l11

Big question, outcomes and success criteria

Big question — Why is a binary search so much faster than a linear search on a long list, and what does using one cost you?

Prerequisites

The lesson before this one is pr-l10. Pupils should already have finished it: the starter below retrieves from it.

Earlier in the unit: pr-l01, pr-l02, pr-l03, pr-l04, pr-l05, pr-l06, pr-l07, pr-l08, pr-l09.

Assessable outcomes

- pr-l11-o1** — know how to code a linear search and a binary search, and report the position each one finds
- pr-l11-o2** — understand how a bubble sort, an insertion sort and a merge sort each put a list in order
- pr-l11-o3** — know how to count the comparisons an algorithm makes, in order to compare two of them

Success criteria the pupil can check against

Taken from the briefs the independent activities themselves show, so these are the words the pupil sees. Their work has met the outcome when it does these things.

- Replace ____ with the comparison that asks whether this item is the one wanted. **pr-l11-s1-a4**
- Display exactly: not found **pr-l11-s1-a4**
- The user types in one whole number. **pr-l11-s1-a5**
- One line of output and nothing else. **pr-l11-s1-a5**
- Count with a loop of your own, not with .count(). **pr-l11-s1-a7**
- The numbers are written into the program, in order, smallest first. **pr-l11-s2-a1**
- The program is looking for 9, which is at position 1. **pr-l11-s2-a3**
- The user types in one whole number, which is the number to find. **pr-l11-s2-a5**

Vocabulary, resources and preparation

Key vocabulary — Linear search, Binary search, Pass, Comparison

Resources

SLIDES	presentations/PR_L11_SearchingAndSorting_Lesson.pptx
WORKSHEET	worksheets/PR_L11_SearchingAndSorting_Worksheet.docx

ONLINE EXPERIENCE	experience.html?id=pr-111
CODE BANK	tools/codebank/pr-111.json

Before the lesson

1. Sign in on the **teacher dashboard** (teacher.html) and open **Class logins**. Enter the class or group and the usernames, one per line, then print the login cards. A pupil cannot sign in without one, so do this before the lesson rather than in it.
2. Print the worksheet, one per pupil.
3. Open the slides and the experience once yourself, on the machine you will be teaching from, so the Python runtime and the 360 image are already cached.
4. Check headsets or devices are charged and on the school network.

How pupils get in

1. Go to topics.html on revise360.co.uk.
2. Sign in with the **username and PIN from the login card**. The class is already set against the card.
3. Choose the topic, then this lesson's card.

Worth knowing before you promise anything. In this build APP_CONFIG.backendUrl is empty, so logins, login cards and progress are stored on the device the pupil used. The teacher dashboard says so too. A pupil who moves to a different machine starts again, so keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels.

Teaching sequence

Sixty minutes including transitions. The online block is the one to move if your lesson is shorter: see the adaptation at the end.

MIN	PHASE	TEACHER	PUPILS	EVIDENCE TO CHECK
0-5	Retrieve	Starter on the board or the worksheet, from the last lesson and this lesson's prerequisites.	Answer the starter on the worksheet.	The starter box on PR_L11_SearchingAndSorting_Worksheet.docx is filled in.
5-8	Review the starter	Take answers; correct the one most got wrong before moving on.	Correct their own starter.	A correction written, not just a tick.
8-16	Explain and model	The new idea, then the worked example, typed live rather than shown finished.	Watch the example being built, and say what each line does.	They can name the line that does the job, not just copy it.
16-19	Set up and settle	Sign-in, launch, and what to record on paper before anyone types.	Sign in, open the lesson, read the task before typing.	Everyone is at station 1 activity 1, not typing yet.
19-48	Online: work the stations in order	Pupils work; you circulate and read the editor over shoulders.	Work the stations in order, running and checking as they go.	The station badges fill in; the editor shows their own program, not the starter.
48-53	Check understanding	The hinge question, hands down, then the reteach if the class splits.	Answer the hinge question, hands down.	The split across the options, before any reteach.
53-58	Apply or extend	The challenge activity, or the exam practice on the worksheet.	The Challenge activity, or the exam practice.	worksheet, Exam practice a) to c)

MIN	PHASE	TEACHER	PUPILS	EVIDENCE TO CHECK
58-60	Exit and set up the next lesson	The exit question and what to finish before next time.	Write the exit answer.	One sentence that answers the big question.

The explanation and the worked example

This is what the walls of the experience say, station by station. Teach the first two or three from the front before anyone puts a headset on; the rest the pupils read for themselves.

1. Linear search

- A linear search checks each item in turn until it finds the one it wants.
- It works on any list, in order or not, and reports the position or that nothing matched.
- Common mistake: printing 'not found' inside the loop, once for every item that did not match.

Challenge on this wall: Why must 'not found' be decided after the loop has finished?

2. Binary search

- A binary search looks at the middle item, then throws away the half that cannot hold the answer.
- It only works on a list that is already in order.
- Common mistake: not moving low or high, which leaves the loop checking the same middle forever.

Challenge on this wall: How much of the list can a binary search rule out with its first comparison?

3. Bubble sort

- A bubble sort compares each neighbouring pair and swaps them if they are the wrong way round.
- Every pass pushes the largest remaining value to the end, so n items need up to $n - 1$ passes.
- Common mistake: a swap that loses a value, because one side was overwritten before it was copied.

Challenge on this wall: How does a bubble sort know that it can stop early?

4. Insertion sort

- An insertion sort takes the next item and slides it back into place among the sorted ones.
- The front of the list is always in order, and it grows by one item each time round.
- Common mistake: overwriting an item instead of shifting it along to make room.

Challenge on this wall: Which part of the list is already sorted part way through an insertion sort?

5. Merge sort

- A merge sort splits the list in half over and over, then merges the pieces back in order.
- Merging compares the front of each sorted list and takes the smaller one each time.
- Common mistake: stopping when one list runs out, instead of copying the rest of the other.

Challenge on this wall: Why is merging two sorted lists easier than sorting one unsorted one?

6. Putting it together

- Count the comparisons to compare two algorithms fairly on exactly the same data.
- Write the search or the sort out yourself, since `sorted()` and `.index()` hide the very steps being counted.
- Common mistake: binary searching a list that was never sorted, which quietly gives the wrong answer.

Challenge on this wall: Plan an experiment that shows a binary search beating a linear one.

The example to model

A line of numbers typed in arrives as one piece of text. `split()` cuts it into a list of parts, and `int()` turns each part into a whole number. Counting the positions with `range(len(nums))` lets a program say where a value sits, not only that it is there.

```
nums = []
for part in input("Numbers? ").split():
    nums.append(int(part))
```

```
for i in range(len(nums)):
    print(i, nums[i])
```

It displays 0 40 / 1 15 / 2 9

Decisions to model out loud, not just the finished answer: say why you chose that name, why that line comes before the next one, and what you would try first if it did not work. The example deliberately uses different data from the task, so a pupil cannot copy it and be finished.

Retrieval and the check for understanding

Starter — The sorted list holds 3, 8, 12, 19, 24, 31, 40. Write down the value a binary search looks at first, second and third while it hunts for 40, and how many comparisons that took.

Answer space: 4 lines on the worksheet.

The answer is in the teacher answers for this unit, not on this plan: a plan left on a desk should not give it away.

Hinge question

Use this one, hands down, before anyone moves on to independent work. It is a real activity from the experience (pr-111-s3-c3), so the wording a pupil sees matches the wording you say.

What will this program display?

```
nums = [4, 2, 7]
checks = 0
for p in range(2):
    for i in range(2):
        checks = checks + 1
        if nums[i] > nums[i + 1]:
            nums[i], nums[i + 1] = nums[i + 1], nums[i]
print(nums)
print(checks)
```

Correct: [2, 4, 7] 4

What a pupil might choose instead

- [2, 4, 7] 2
- [2, 4, 7] 6

Count the hands on each. If more than a few of the class are on one wrong option, that option is the misconception to teach against before anyone goes on: go back to the station this question sits on and work through its example again, aloud, before the independent work.

The course's own response to a wrong answer here: Four comparisons for three numbers. A bubble sort does far more comparisons than a search, which is why sorting costs so much more.

One response, not four. one response covers every wrong option; it does not say what to revisit for each one, so decide your own reteach for each option before the lesson - the list above is the start of it.

Misconceptions this lesson addresses

- printing 'not found' inside the loop, once for every item that did not match. [wall panel](#)
- The program cannot know a number is missing until the whole loop has finished. [pr-111-s1-a6, a Fix it activity](#)
- not moving low or high, which leaves the loop checking the same middle forever. [wall panel](#)
- A position has to be a whole number, and only one of Python's two divide operators gives a whole number. [pr-111-s2-a7, a Fix it activity](#)
- a swap that loses a value, because one side was overwritten before it was copied. [wall panel](#)
- One pass only carries the largest value to the end. Something has to repeat the whole pass. [pr-111-s3-a6, a Fix it activity](#)
- overwriting an item instead of shifting it along to make room. [wall panel](#)

- Which way does a bigger item have to move, and which side of the = is the place it is moving to?

pr-l11-s4-a6, a Fix it activity

Guided to independent practice

STAGE	ACTIVITIES	WHAT IT IS
Recognise	19	The answer is on the screen and the pupil picks it: multiple choice, choosing several, matching a pair.
Construct	8	Nothing is offered. The pupil classifies, orders, fills in a table or works a number out, and types or places the answer.
Produce	12	The pupil writes it from the brief and nothing else.
Challenge	6	The hardest activity on each station. It is not optional and it is not extra: it is the last step of the station.

Scaffolds already in the product, and how they fade

- **Hint** is a ladder, not a door: the first rung names the idea, the next shows the shape of the code, the next how it starts, the last animates the technique on different data. A pupil asks for each rung. None of them is the answer.
- **Run** is free and unmarked; **Check my answer** marks. A pupil may run as often as they like, which is what makes trial and error safe.
- **I need help** says to ask you, and says plainly that the question still has to be completed. It does not notify anyone and it does not unlock anything.
- The worked example appears on every early activity and on none of the late ones. That fade is deliberate; it is not a missing example.

Accessibility

- **Read aloud** on every task window reads the task and its brief, with the prescribed values read as values rather than as punctuation.
- Text is 16px or larger throughout, and a question window scrolls rather than shrinking its type.
- Prescribed data is boxed and in a monospace face as well as coloured, so it still reads as code in black and white, in a high-contrast mode, and to a pupil who does not separate green from amber.
- **Lower literacy and EAL:** pre-teach the vocabulary above; let the pupil say the key fact before writing it; accept the key fact in two or three words.
- **SEND:** the sequence is compulsory but the pace is not. A pupil may finish a station next lesson; the progress saves after every answer and reopens at their earliest unfinished question.
- A headset is never required. Every experience works on a laptop or a tablet with a mouse or a finger.

Delivering the online experience

Before anyone enters

- Worksheet out, pen in hand. The rule is write first, then answer at the badge: the first answer is the one that counts.
- Say where the stations are. They are numbered around the room: 1 and 2 to the right, 3 and 4 behind, 5 and 6 to the left, and the final challenge on the floor.
- Say that a badge marked **Not yet** is not a fault. It opens when the station they are on is finished.

The stations, and what to record at each

#	STATION	ACTIVITIES	RECORD ON PAPER
1	Linear search	7	A linear search over a million items may need a million comparisons, which is why sorted data is...
2	Binary search	8	Doubling the length of the list adds only one more comparison, so a million items need about...
3	Bubble sort	7	A whole pass that makes no swaps at all means the list is already in order, so the sort can...

#	STATION	ACTIVITIES	RECORD ON PAPER
4	Insertion sort	7	An insertion sort is how most people put a hand of cards in order without ever being...
5	Merge sort	8	A merge sort usually beats a bubble sort on a long list, but what it costs you is extra...
6	Putting it together	7	Sorting costs time too, so for one single search on unsorted data the winner can still be a...
final	Final challenge: which algorithm?	1	the answer to the station's challenge

Where to pause

Stop the class after station 2 and take answers to its challenge out loud. That is the point where a pupil who has misread the technique can still be put right cheaply.

Afterwards

Headsets off and down. Ask three pupils for the key fact from a station of your choosing, in their own words and not read off the sheet. Then the exit question.

Python: the order is not negotiable. Each activity teaches what the next one needs, so a pupil works through them in order and completes every one. A shorter lesson, homework or a slow worker does not authorise skipping: the pupil stops where they are and resumes at their earliest unfinished question next time, because progress saves after every answer.

A paper activity, a help request or a tick on this plan does not mark an online question complete, and there is no teacher bypass. If a pupil is stuck after the hint, that is the moment to teach them, not to move them on.

Assessment, evidence and unfinished work

ID	QUESTION	MARKS	WHERE
pr-111-x1	A linear search prints Not found from inside its loop, so one search prints that message several times. State where the message should go instead, and explain why putting it there fixes the problem.	3	worksheet, Exam practice a); deck, Exam practice slide
pr-111-x2	Write a program that reads eight whole numbers into a list, puts them in order smallest first with a bubble sort of your own, and prints the sorted list on one line.	6	worksheet, Exam practice b); deck, Exam practice slide
pr-111-x3	Write a program that reads a whole number and finds it in a list of ten numbers that is already in order, using a binary search, and prints its position or Not found.	6	worksheet, Exam practice c); deck, Exam practice slide

Expected answers

Model answers and marking guidance are teacher-only and are not printed on this plan. They are in the unit's teacher guide pack, and the per-activity marking for this lesson is in `answers/records/pr-111.json`, which is not published with the website.

Every programming activity in this lesson is marked by running the pupil's program against its test cases, so the mark is the program's behaviour and not its appearance. The model solutions are in `answers/codebank/pr-111.json`.

Common errors

The wrong options the activities offer are the errors this lesson expects. The commonest to watch for:

- **What will this program display?** — 1; 3
- **What will this program display?** — found at 2 checks: 3; found at 2 checks: 2
- **The program is looking for 9. What will it display?** — 0 3; 4 6
- **The program displays every middle it looks at. What will it display?** — 3 2 1; 0 1 2

- The program is looking for 20, which is not in the list. What will it — -1 4; -1 7

Unfinished work

Record where each pupil stopped, not a tick. They resume at their earliest unfinished question, and a station left half-done is the thing to open next lesson. Do not mark the lesson complete for a pupil who did not finish it: the score screen already shows what was and was not done, and that is the honest record.

Exit, next lesson and homework

Exit check

One of these, written, not a show of hands:

- Coding a linear search and a binary search
- Coding a bubble, insertion or merge sort
- Explaining why a binary search needs sorted data

The confidence columns on the worksheet are a supplement to this, not a substitute: a pupil who rates themselves confident and cannot answer the exit question has told you nothing.

Retrieval link to next lesson

Next is pr-112, Validation and authentication. Use this lesson's big question as next lesson's starter, so the first thing they do is retrieve it.

Homework

Finish any station left open, on the website, from the question they stopped at. If the class has no devices at home, set the exam practice on the worksheet instead and mark it next lesson.

Shorter lessons, longer lessons and no internet

If this lesson takes more than an hour

It will. There are 45 activities across 6 stations, and a beginner does not finish 45 of them in twenty-nine minutes. Plan it as two or three sessions of an hour: stations 1 and 2 in the first, 3 and 4 in the second, 5, 6 and the final challenge in the third. Each session keeps the same starter-explain-model opening; the online block resumes where the class got to. Honest total: two to three hours.

A 40 to 45 minute lesson

Keep the starter, the explanation and the model at full length: cutting those is what makes the online work fail. Take the time out of the online block and the apply phase. Pupils stop where they are; their progress saves after every answer and reopens at their earliest unfinished question, so the next lesson starts there.

A shorter lesson still does not authorise skipping a question. The save-and-resume point is wherever the pupil got to; there is no way to mark a question complete without completing it, and no teacher bypass.

If the internet or the devices are unavailable

Teach from the slides and the worksheet. The slides carry the same teaching as the station walls, and the worksheet carries the key facts, the challenges and the exam practice. Set the online stations when the devices are back.

There is no offline version of the experience, and this plan does not contain one. Do not hand out a printed walkthrough of the stations as a substitute: the experiences are used on the website.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.